

STAT462_Assignment2

Wenjuan Wang

2025-08-30

Question A. Automatically detecting seed types (Logistic Regression)

A.1 Load the datasets `seeds_training.csv` and `seeds_test.csv`. Since the class labels (the seed types) contain characters that R struggles with, encode the class features as a factor variable in both training and test set.

```
# Load the seeds training and testing csv files
seeds_training <- read.csv("seeds_training.csv")
seeds_testing <- read.csv("seeds_test.csv")

# Set the levels for encoding
levs <- c("Cercevelik", "Urgup Sivrisi")

# Encode the 'Class' column into factors and store it in a new column 'Class_factor'
seeds_training$Class_factor <- factor(seeds_training$Class, levels = levs)
seeds_testing$Class_factor <- factor(seeds_testing$Class, levels = levs)

# Output some information about the datasets
cat("The dimensions of seeds_training are: ", dim(seeds_training), "\n")
```

```
## The dimensions of seeds_training are: 2000 14
```

```
cat("The dimensions of seeds_testing are: ", dim(seeds_testing), "\n")
```

```
## The dimensions of seeds_testing are: 500 14
```

```
cat("The two classes in seeds_training are: ", unique(seeds_training$Class), "\n")
```

```
## The two classes in seeds_training are: Urgup Sivrisi Cercevelik
```

A.2 Fit a multiple logistic regression classifier using all of the features and record its accuracy on the test set.

```
# Fit a multiple logistic regression model
# Not including the repeated column 'Class'
m_logistic_model <- glm(Class_factor ~ . - Class, data = seeds_training, family = binomial)

# Calculate the probability of Class "Urgup Sivrisi"
pred_prob <- predict(m_logistic_model, newdata = seeds_testing, type = "response")

# According to the probability, assign the predicted class for each entry
```

```

pred_classes <- ifelse(pred_prob >= 0.5, levs[2], levs[1])

# Compute the accuracy on testing dataset
accuracy <- mean(pred_classes == seeds_testing$Class_factor)
paste("The accuracy on testing dataset is: ", round(accuracy,4))

```

```
## [1] "The accuracy on testing dataset is: 0.874"
```

Therefore, the accuracy predicted using the logistic model above based on the testing dataset is 0.874.

A.3 Provide a confusion matrix for your model based on its predictions on the test set. Why does the concept of sensitivity and specificity not make a lot of sense here?

```

# Compute the confusion matrix using the test dataset based on the predicted classes in A.2.
confusionMatrix(as.factor(pred_classes),as.factor(seeds_testing$Class_factor),
                positive = levs[1])

```

```

## Confusion Matrix and Statistics
##
##              Reference
## Prediction      Cercevelik Urgup Sivrisi
## Cercevelik           242         44
## Urgup Sivrisi         19        195
##
##              Accuracy : 0.874
##              95% CI : (0.8417, 0.9018)
##      No Information Rate : 0.522
##      P-Value [Acc > NIR] : < 2.2e-16
##
##              Kappa : 0.7464
##
## Mcnemar's Test P-Value : 0.002497
##
##              Sensitivity : 0.9272
##              Specificity : 0.8159
##      Pos Pred Value : 0.8462
##      Neg Pred Value : 0.9112
##              Prevalence : 0.5220
##      Detection Rate : 0.4840
##      Detection Prevalence : 0.5720
##      Balanced Accuracy : 0.8716
##
##      'Positive' Class : Cercevelik
##

```

The two classes are simply two seed types of equal importance; neither is inherently “positive” or “negative.” Because sensitivity and specificity are defined relative to whichever class is designated as positive (e.g., “loan default” vs. “no default”), they are somewhat arbitrary here and thus less informative.

A.4 Consulting the summary of your glm model, which three features are most significant?

```
# Summarize the logistic model.
sum_model = summary(m_logistic_model)

# Find the three most significant features with the lowest p-values in coefficients.
coefs <- as.data.frame(sum_model$coefficients)
coefs = coefs[rownames(coefs) != "(Intercept)", ,drop = FALSE]
coefs = coefs["Pr(>|z|)"]
top3_idx <- order(coefs[["Pr(>|z|)"]])[1:3]
top3 <- coefs[top3_idx, , drop = FALSE]
top3
```

```
##              Pr(>|z|)
## Equiv_Diameter 3.040790e-09
## Minor_Axis_Length 2.705284e-04
## Area          9.922781e-03
```

According to the p-values in the summary, “Minor_Axis_Length”, “Equiv_Diameter” and “Area” are the most significant because they have the lowest three p-values that are below 0.01.

A.5 Train another glm model, but this time using only the three most significant features from your first model run. Compute test set accuracy and a confusion matrix in the same way as for your first model and briefly compare the two models. Which model would you pick, and why?

```
# Fit a multiple logistic regression model with only 3 predictors selected in A.4.
m_logistic_model_new <- glm(Class_factor ~ Area + Minor_Axis_Length + Equiv_Diameter,
                             data = seeds_training, family = binomial)
summary(m_logistic_model_new)
```

```
##
## Call:
## glm(formula = Class_factor ~ Area + Minor_Axis_Length + Equiv_Diameter,
##      family = binomial, data = seeds_training)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)  93.3898356 11.1193184   8.399  < 2e-16 ***
## Area         0.0012058  0.0001391   8.667  < 2e-16 ***
## Minor_Axis_Length -0.1863422  0.0083172 -22.405  < 2e-16 ***
## Equiv_Diameter  -0.4651483  0.0683952  -6.801 1.04e-11 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 2769.5  on 1999  degrees of freedom
## Residual deviance: 1224.8  on 1996  degrees of freedom
## AIC: 1232.8
##
## Number of Fisher Scoring iterations: 6
```

```

# Calculate the probability of Class "Urgup Sivrisi"
pred_prob <- predict(m_logistic_model_new, newdata = seeds_testing, type = "response")

# According to the probability, assign the predicted class for each entry
pred_classes <- ifelse(pred_prob >= 0.5, levs[2], levs[1])

# Accuracy: proportion of correct prediction
accuracy <- mean(pred_classes == seeds_testing$Class_factor)
paste("The accuracy on testing dataset is: ", round(accuracy,4))

## [1] "The accuracy on testing dataset is: 0.878"

# Compute the confusion matrix for new model with 3 predictors
confusionMatrix(as.factor(pred_classes), as.factor(seeds_testing$Class_factor),
                positive = levs[2])

## Confusion Matrix and Statistics
##
##              Reference
## Prediction   Cercevelik Urgup Sivrisi
## Cercevelik      237      37
## Urgup Sivrisi    24      202
##
##              Accuracy : 0.878
##              95% CI : (0.8461, 0.9054)
##      No Information Rate : 0.522
##      P-Value [Acc > NIR] : <2e-16
##
##              Kappa : 0.755
##
##  Mcnemar's Test P-Value : 0.1244
##
##      Sensitivity : 0.8452
##      Specificity : 0.9080
##      Pos Pred Value : 0.8938
##      Neg Pred Value : 0.8650
##      Prevalence : 0.4780
##      Detection Rate : 0.4040
##      Detection Prevalence : 0.4520
##      Balanced Accuracy : 0.8766
##
##      'Positive' Class : Urgup Sivrisi
##

```

Compared with the first model that uses all 12 variables, the second model attains slightly higher accuracy (0.878 vs 0.874) while using only one quarter as many predictors. Moreover, all three predictors in the second model are statistically significant. Therefore, I prefer the second model for its parsimony and marginally better accuracy.

Question B. Predicting color name from RGB values (Discriminant Analysis)

B.1. Load the dataset `colors_train.csv`. How many classes are there in the dataset?

```
# Load colors_train.csv
color_train <- read.csv("colors_train.csv")
color_test  <- read.csv("colors_test.csv")

# There are three columns in the csv file and get the unique values in 'color' column
uni_colors <- unique(color_train$color)
uni_colors
```

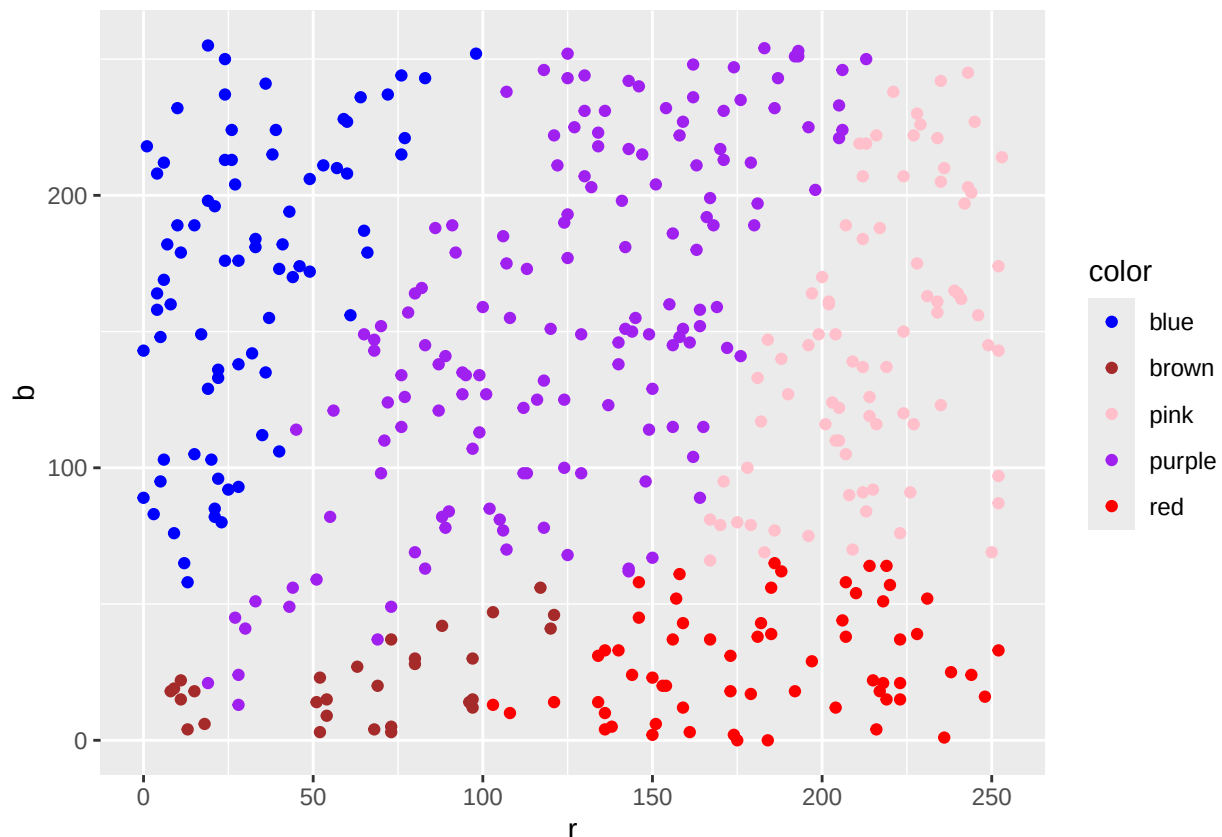
```
## [1] "pink" "purple" "red" "brown" "blue"
```

There are a total of 5 classes in the `colors_train.csv`.

Visualize the `color_train` dataset.

```
# Set the symbol of the points corresponding to 'color' attributes
lvls <- c("blue", "brown", "pink", "purple", "red")
pal  <- c(blue="blue", brown="brown", pink="pink", purple="purple", red="red")

# Visualization
ggplot(data=color_train) +
  geom_point(aes(x=r, y=b, color=color), alpha = 1) +
  scale_colour_manual(values = pal, breaks = lvls, labels = lvls)
```



B.2. Fit a QDA algorithm to this classification problem and compute the confusion matrix.

```
# Fit the QDA model with train dataset
# 'r' and 'b' are predictors while 'color' is the dependent variable.
qda.fit <- qda(color ~ r + b, data = color_train)

# Make predictions on the test dataset.
# Use $class to extract the predicted classes
pred.qda.fit = predict(qda.fit, newdata = color_test)$class

# Retrieve the actual colors in test dataset for comparison
actual <- color_test$color

# Make a confusion matrix to evaluate the model
confusionMatrix(as.factor(pred.qda.fit), as.factor(actual))
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction blue brown pink purple red
##   blue      15     0     0      0  0
##   brown      0    10     0      1  0
##   pink        0     0    22      4  0
##   purple      0     0     0     31  0
##   red         0     0     0      0 17
##
## Overall Statistics
##
##               Accuracy : 0.95
##               95% CI : (0.8872, 0.9836)
##   No Information Rate : 0.36
##   P-Value [Acc > NIR] : < 2.2e-16
##
##               Kappa : 0.935
##
##  McNemar's Test P-Value : NA
##
## Statistics by Class:
##
##               Class: blue Class: brown Class: pink Class: purple
## Sensitivity              1.00          1.0000          1.0000          0.8611
## Specificity              1.00          0.9889          0.9487          1.0000
## Pos Pred Value           1.00          0.9091          0.8462          1.0000
## Neg Pred Value           1.00          1.0000          1.0000          0.9275
## Prevalence               0.15          0.1000          0.2200          0.3600
## Detection Rate           0.15          0.1000          0.2200          0.3100
## Detection Prevalence     0.15          0.1100          0.2600          0.3100
## Balanced Accuracy        1.00          0.9944          0.9744          0.9306
##
##               Class: red
## Sensitivity              1.00
## Specificity              1.00
## Pos Pred Value           1.00
## Neg Pred Value           1.00
```

```
## Prevalence          0.17
## Detection Rate      0.17
## Detection Prevalence 0.17
## Balanced Accuracy   1.00
```

The confusion matrix shows 95% overall accuracy ($Kappa = 0.935$). All errors occur in the purple class: 1 instance predicted as brown and 4 as pink. Misclassifications cluster near the purple–pink and purple–brown decision boundaries.

B.3. Visualise the decision boundaries in a suitable way.

```
# Create a grid for interpolating the decision boundary.
# Compute a sequence of 300 numbers within the range of r and b, respectively
r_seq <- seq(min(color_train$r), max(color_train$r), length.out = 300)
b_seq <- seq(min(color_train$b), max(color_train$b), length.out = 300)
grid <- expand.grid(r = r_seq, b = b_seq)

# Use the QDA model fitted in B.2 to predict the color of each point.
prd <- predict(qda.fit, newdata = grid)

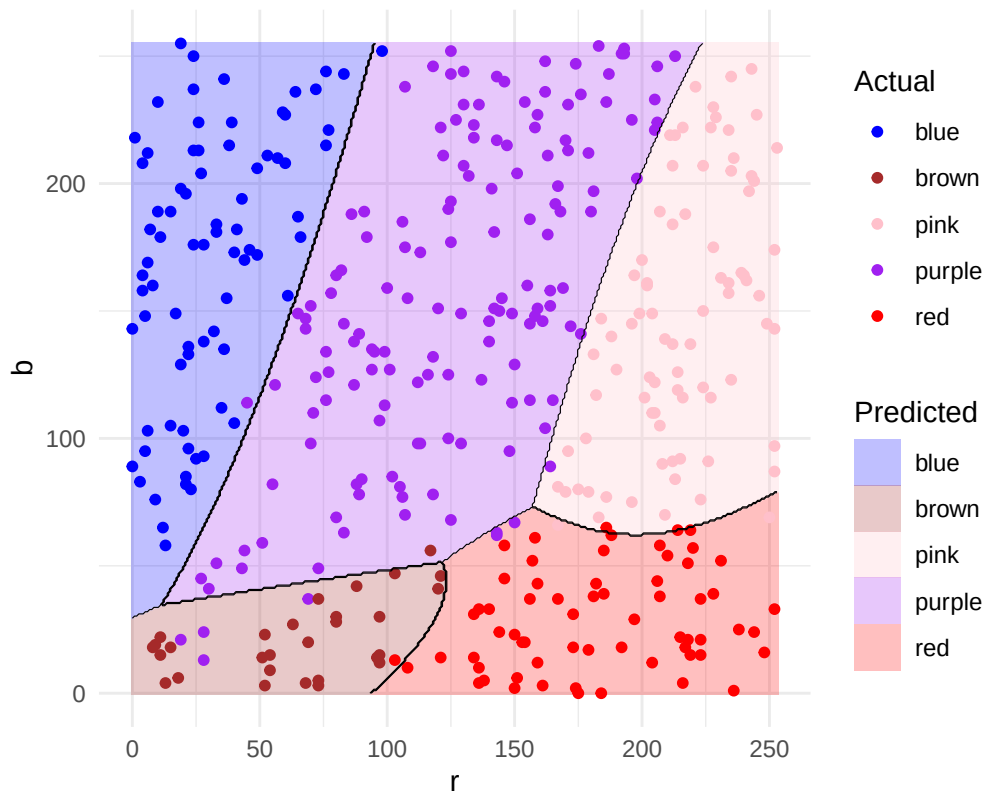
# Store the predicted classes and corresponding posteriors in grid
grid$pred <- factor(prd$class, levels = lvls)
post <- as.data.frame(prd$posterior); names(post) <- lvls
grid <- cbind(grid, post)

# Visualize the points and filled color decision boundaries
p <- ggplot() +
  # Set the filled colors
  geom_raster(data = grid, aes(r, b, fill = pred), alpha = 0.25, interpolate = TRUE) +
  scale_fill_manual(values = pal, name = "Predicted") +
  # Add the observation points in train dataset
  geom_point(data = color_train, aes(r, b, colour = color), size = 1.4) +
  scale_colour_manual(values = pal, name = "Actual") +
  coord_equal() + theme_minimal() +
  # Set the title and its position in the middle
  labs(title = "The QDA Decision Boundaries of the Classification Model", x = "r", y = "b") +
  theme(plot.title = element_text(hjust = 0.5))

# Covert the predicted_id (the order in the color levels) into integer
grid$pred_id <- as.integer(grid$pred)

# Add the decision boundary lines
p + stat_contour(
  data = grid,
  aes(r, b, z = pred_id),
  # Add 0.5 to each classes for more appropriate match between the lines and filled color edges
  breaks = seq(1.5, length(lvls) - 0.5, by = 1),
  colour = "black", linewidth = 0.2
)
```

The QDA Decision Boundaries of the Classification Model



The QDA decision regions match the B.2 confusion matrix. Accuracy is high (95%), and most observations are correctly classified; the only errors are a few purple samples misclassified as pink or brown, mainly along the class boundaries.

B.4. Test your algorithm on (200,0,200). What colour is this being called by your algorithm?

```
# Create a dataframe to store the test data
test <- data.frame(r = 200, b = 200)

# Use the QDA model to predict the test data
test_pre <- predict(qda.fit, newdata = test)

# Output the predicted class.
print(test_pre$class)

## [1] pink
## Levels: blue brown pink purple red

# Output the posteriors of the test point (decending)
sort(test_pre$posterior[1, ], decreasing = TRUE)

##          pink          purple          blue          red          brown
## 5.538038e-01 4.461962e-01 7.145960e-15 8.530348e-18 2.072491e-35
```

The test data(200,0,200) is classified as **pink** using the QDA model.

The top-1 posterior (pink = 0.554) and top-2 posterior (purple = 0.446) are close, while the other classes

have near-zero probability($7.145960\text{e-}15$, $8.530348\text{e-}18$, $2.072491\text{e-}35$). This indicates the point lies near the pink–purple decision boundary.